

Projekt: Rozpoznawania liter

Michał Nowakowski

s30965

16c

Małgorzata Świderek

s31302

16c

Styczeń 2026

Spis treści

1	Cel badania i opis zbioru danych	3
2	Metodologia i rozwiązanie	3
2.1	k-Najbliższych Sąsiadów (k-NN)	3
2.2	Klasyfikator Bayesowski (Naive Bayes)	3
2.3	Las Losowy (Random Forest)	4
3	Wstępne przetwarzanie danych	4
4	Metoda oceniania jakości modelu	4
5	Wyniki eksperymentalne	5
6	Podsumowanie	9

1 Cel badania i opis zbioru danych

Stworzenie trzech klasyfikatorów do rozpoznawania liter, zmierzenie miar oceny na wytrenowanych modelach, porównanie ich względem siebie i wyciągnięcie wniosków.

Źródło danych: Dane pochodzą z repozytorium UCI Machine Learning Repository (zbiór *Letter Recognition*).

Szczegóły zbioru danych:

- **Liczba atrybutów :** 16 atrybutów numerycznych (liczby całkowite z zakresu 0-15).
- **Liczba rekordów :** 20 000 przykładów.
- **Rozkład klas decyzyjnych:** 26 klas (litery alfabetu angielskiego od A do Z).

2 Metodologia i rozwiązanie

W badaniu wybrano i porównano trzy klasyfikatory:

2.1 k-Najbliższych Sąsiadów (k-NN)

Algorytm ten klasyfikuje nowy obiekt na podstawie klasy, do której należy większość z jego k najbliższych sąsiadów w przestrzeni cech. W eksperymencie sprawdzono skuteczność dla wartości parametru k (od 1 do 10).

- Wykorzystywany najczęściej dla danych z atrybutami numerycznymi.
- Za pomocą funkcji odległości dopasowanej do danych wyznaczane jest k najbliższych sąsiadów nowego obiektu, aby następnie na podstawie ich decyzji wyznaczyć jego klasę.
- K-nn jest przykładem metody uczenia leniwego, ponieważ nie buduje jawnego modelu wiedzy.
- Za małe k spowoduje zwiększoną wrażliwość klasyfikatora na drobne szumy w danych, a za duże k spowoduje wysoką złożoność obliczeniową.
- Metoda wykazuje najlepsze wyniki dla danych z małą liczbą atrybutów co jest korzystne dla danych zawartych w projekcie.
- Złożoność czasowa treningu: $O(1)$.
- Złożoność czasowa predykcji: $O(nd)$ (n - liczba przykładów w zbiorze treningowym, d - liczba cech).

2.2 Klasyfikator Bayesowski (Naive Bayes)

Zastosowano naiwny klasyfikator Bayesa (Gaussian Naive Bayes). Metoda ta opiera się na twierdzeniu Bayesa i zakłada niezależność atrybutów.

- Wersja klasyfikatora Bayesa wykorzystywana w przypadku cech ciągłych (przyjmuje, że ich rozkłady w obrębie każdej klasy mają charakter normalny Gaussa).
- Klasyfikator oparty jest na twierdzeniu Bayesa naiwnie zakładającym niezależność cech od siebie nawzajem.
- Klasyfikacja polega na wyliczeniu prawdopodobieństw posteriori przynależności obiektu do klas i wyborze największego z nich.
- Metoda wykazuje dobrą skuteczność przy dużej ilości cech, w naszym zbiorze danych cech jest mało więc zaobserwujemy zmniejszoną skuteczność klasyfikatora.
- Złożoność czasowa treningu: $O(nd)$ (n - liczba przykładów w zbiorze treningowym, d - liczba cech).
- Złożoność czasowa predykcji: $O(cd)$ (c - liczba klas, d - liczba cech).

2.3 Las Losowy (Random Forest)

Algorytm ten należy do grupy metod zespołowych. Polega na budowie wielu drzew decyzyjnych, każde drzewo podejmuje własną decyzję, a najczęściej pojawiająca się decyzja staje się tą właściwą. Parametry, które zostały wykorzystane do optymalizacji modelu to maksymalna głębokość drzewa(max_depth) oraz kryterium podziału (gini, entropy, log_loss).

- Wykorzystuje technikę agregacji (bagging) oraz losowy wybór podzbioru cech dla każdego węzła, co pozwala na budowę nieskorelowanych ze sobą drzew.
- Klasyfikacja końcowa odbywa się na drodze głosowania większościowego - obiekt zostaje przypisany do klasy, która została wskazana przez największą liczbę drzew w lesie.
- Jest przykładem metody uczenia goriwego, ponieważ w fazie treningu buduje jawny model (zbiór drzew).
- Algorytm charakteryzuje się wysoką stabilnością i odpornością na szumy w danych, co przekłada się na wysoką precyzję.
- Złożoność czasowa treningu: $O(n_{trees} \cdot d \cdot n \log n)$ (n_{trees} - liczba drzew, d - liczba cech, n - liczba przykładów w zbiorze treningowym).
- Złożoność czasowa predykcji: $O(n_{trees} \cdot depth)$ (n_{trees} - liczba drzew, $depth$ - głębokość drzewa).

3 Wstępne przetwarzanie danych

Dane zostały znormalizowane za pomocą **normalizacji min-max**. Następnie **16000** pierwszych liter (wierszy) zostało przydzielone do **zbioru treningowego**, a pozostałe **4000** do **zbioru testowego**.

4 Metoda oceniania jakości modelu

Za najważniejszą miarę przyjęto **dokładność (Accuracy)**. Dla zwiększenia dokładności oceny wyznaczono również miary takie jak: **czas treningu z predykcją (Time)**, **precyzję (Precision)**, **czułość (Recall)**, **wsparcie (Support)**, **f-miarę (F-Measure)**, **macierz pomyłek (Confusion Matrix)**.

5 Wyniki eksperymentalne

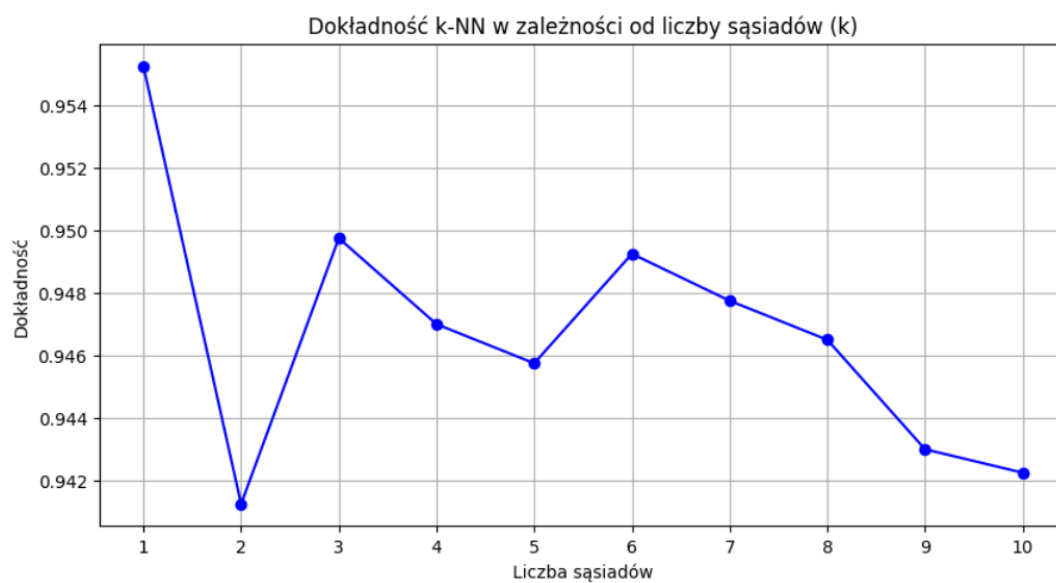


Figure 1: Dokładność k-NN w zależności od liczby sąsiadów (k)

Na załączonym wykresie "Dokładność k-NN w zależności od liczby sąsiadów (k)" zawarta jest informacja o optymalnej liczbie sąsiadów dla modelu k-NN. Widać spadek dokładności algorytmu po zwiększeniu ilości sąsiadów.

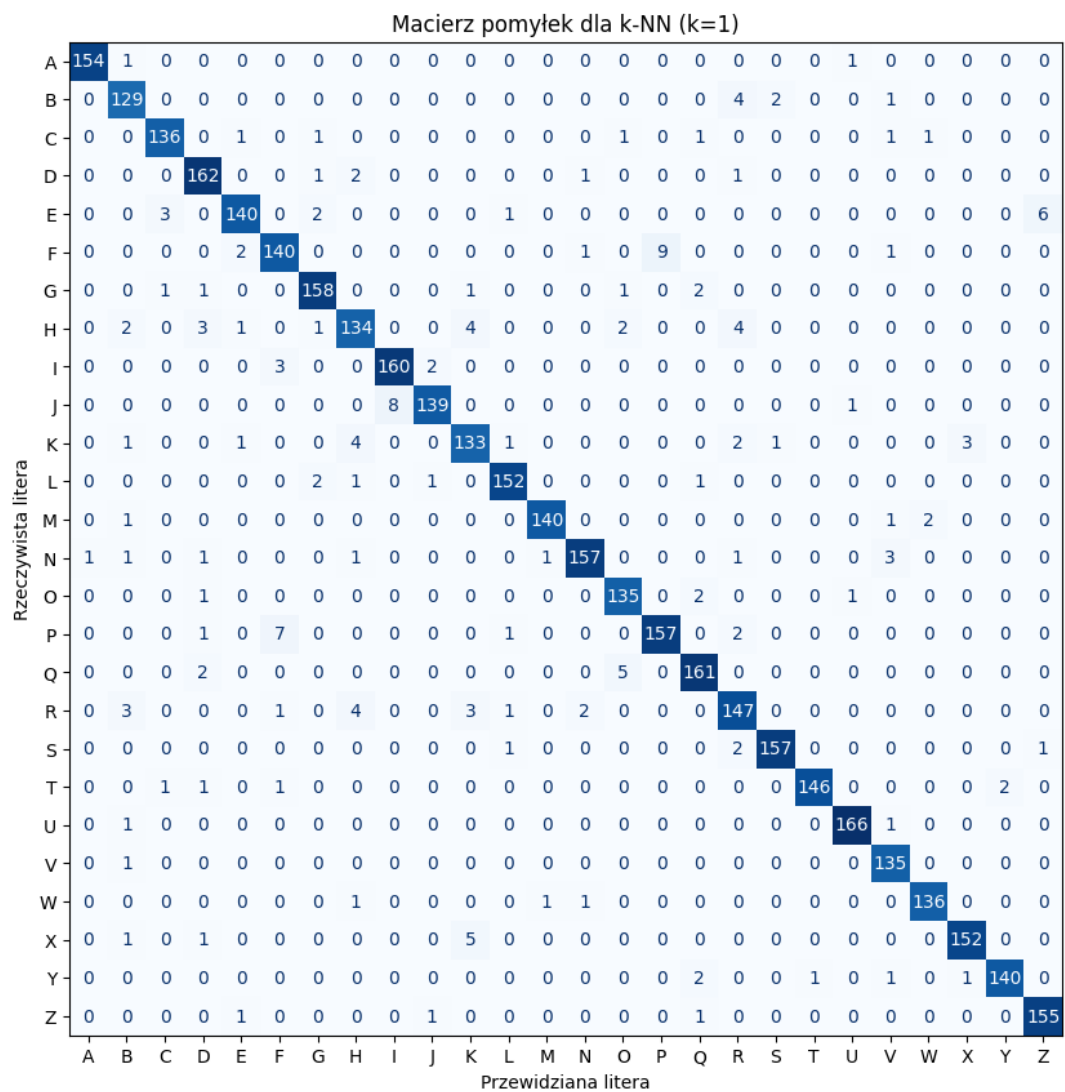


Figure 2: Macierz pomyłek dla k-NN

Na załączonym wykresie "Macierz pomyłek dla k-NN" widzimy wysoką dokładność modelu (ciemny kolor widoczny jest jedynie na przekątnej macierzy).

Macierz pomyłek																												
Rzeczywista litera	A	131	0	0	1	0	0	0	2	0	0	0	0	5	1	0	0	3	0	7	0	1	0	1	1	3	0	
	B	0	90	0	9	0	1	0	1	17	0	1	0	3	0	0	0	0	13	0	0	0	0	0	1	0	0	
	C	0	0	104	0	3	0	7	0	0	0	16	0	4	0	3	0	2	0	0	1	1	1	0	0	0	0	
	D	0	12	0	115	0	0	0	0	13	4	1	0	0	0	11	1	0	6	4	0	0	0	0	0	0	0	
	E	0	2	1	0	58	0	26	0	10	0	7	0	0	0	0	0	9	0	7	2	0	0	0	24	0	6	
	F	0	7	0	5	0	112	5	0	1	0	0	0	0	3	0	9	1	1	1	2	0	0	2	1	3	0	
	G	1	4	26	3	0	1	94	0	2	0	3	0	2	0	2	0	12	3	4	0	0	0	7	0	0	0	
	H	1	4	0	13	0	5	2	41	1	0	10	0	5	2	27	0	1	11	1	0	3	0	3	19	2	0	
	I	0	4	0	6	3	3	0	0	129	8	0	1	0	0	0	2	2	0	6	0	0	0	0	0	0	1	
	J	0	4	0	9	0	3	0	0	5	110	0	0	0	0	1	2	0	1	11	0	0	0	0	2	0	0	
	K	0	7	1	5	14	0	5	0	0	0	66	0	3	2	0	0	1	19	0	2	3	0	0	17	1	0	
	L	0	4	0	0	1	0	6	0	0	9	8	117	0	0	0	0	6	2	1	0	0	0	0	3	0	0	
	M	3	2	0	0	0	0	0	2	0	0	4	0	129	0	0	0	0	1	0	0	0	0	3	0	0	0	
	N	2	2	0	5	0	0	0	21	1	0	3	0	5	95	9	1	0	7	0	0	2	6	7	0	0	0	
	O	2	1	1	6	0	0	9	1	7	0	3	0	1	1	93	1	4	7	0	0	0	0	2	0	0	0	
	P	0	3	0	9	0	11	5	2	0	0	0	0	0	2	0	121	0	0	1	0	0	1	9	0	4	0	
	Q	5	2	0	3	0	0	2	0	4	0	1	2	3	0	29	0	93	4	16	0	0	0	2	1	1	0	
	R	0	13	0	13	0	0	0	5	4	7	3	0	5	1	3	0	1	105	0	0	0	0	1	0	0	0	
	S	11	24	1	1	4	4	3	0	13	1	2	1	0	0	0	0	7	2	45	5	1	1	0	18	1	16	
	T	0	0	0	1	1	9	6	0	0	0	7	0	0	0	0	0	2	3	104	0	4	0	5	8	1		
	U	0	0	3	2	0	0	2	5	0	0	8	0	15	4	10	0	2	0	0	0	114	0	2	1	0	0	
	V	0	4	0	0	0	2	1	0	0	0	0	0	2	1	0	3	0	0	1	0	0	105	15	0	2	0	
	W	0	2	0	0	0	0	0	2	0	0	0	0	9	2	3	0	0	0	0	0	14	107	0	0	0		
	X	0	9	0	4	1	0	0	0	12	0	9	0	0	0	25	1	0	0	5	4	3	0	0	80	1	5	
	Y	0	0	0	3	0	10	0	0	0	0	0	0	3	0	0	1	4	0	7	22	2	39	4	0	50	0	
	Z	1	0	0	1	1	3	0	0	16	2	2	2	0	0	0	0	1	2	25	3	0	0	0	5	1	93	
	Przewidywana litera																											

Figure 3: Macierz pomyłek dla Bayes

Na załączonym wykresie "Macierz pomyłek dla Bayes" widzimy niską dokładność modelu. W odróżnieniu od "Macierz pomyłek dla k-NN" ciemne odcienie niebieskiego znajdują się również poza przekątną, a sama przekątna posiada jasne odcienie niebieskiego. Oznacza to niską dokładność modelu w rozpoznawaniu tych liter. Możemy zauważyć duży spadek dokładności między literą A a literą H.

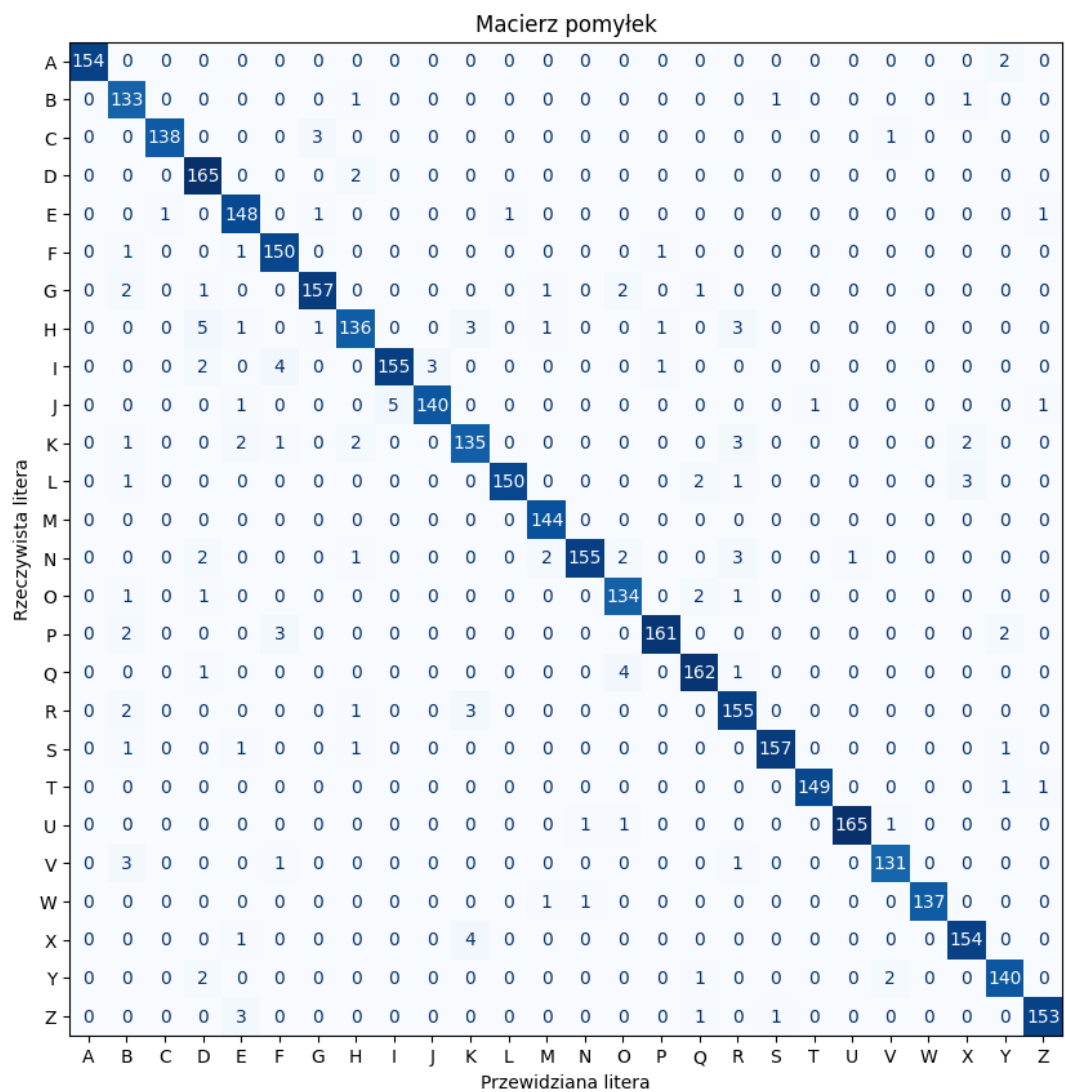


Figure 4: Macierz pomyłek dla Random Forest

Na załączonym wykresie "Macierz pomyłek dla Random Forest" widzimy wysoką i minimalnie lepszą od k-NN dokładność modelu (ciemny kolor widoczny jest jedynie na przekątnej macierzy).

Metryka / Litera	Bayes	K-NN	Random Forest
Accuracy (główna miara)	62.52%	95.53%	96.45%
Czas uczenia i predykcji	0.04s	0.662s	2.36s
Precision	0.64	0.96	0.97
Recall	0.63	0.96	0.96
F1 - score	0.62	0.96	0.96
Support	4000	4000	4000
Accuracy dla każdej z liter			
A	83.97%	98.72%	98.72%
B	66.18%	94.85%	97.79%
C	73.24%	95.77%	97.18%
D	68.86%	97.01%	98.80%
E	38.16%	92.11%	97.37%
F	73.20%	91.50%	98.04%
G	57.32%	96.34%	95.73%
H	27.15%	88.74%	90.07%
I	78.18%	96.97%	93.94%
J	74.32%	93.92%	94.59%
K	45.21%	91.10%	92.47%
L	74.52%	96.82%	95.54%
M	89.58%	97.22%	100.00%
N	57.23%	94.58%	93.37%
O	66.91%	97.12%	96.40%
P	72.02%	93.45%	95.83%
Q	55.36%	95.83%	96.43%
R	65.22%	91.30%	96.27%
S	27.95%	97.52%	97.52%
T	68.87%	96.69%	98.68%
U	67.86%	98.81%	98.21%
V	77.21%	99.26%	96.32%
W	76.98%	97.84%	98.56%
X	50.31%	95.60%	96.86%
Y	34.48%	96.55%	96.55%
Z	58.86%	98.10%	96.84%

6 Podsumowanie

Należy zauważyć, że klasyfikator **Bayesa** znacząco odstaje jakością od pozostałych klasyfikatorów. Dzieje się tak, ponieważ nie jest to klasyfikator przeznaczony do tego typu danych i jego gorszy wynik jest oczekiwany. Wiedząc to, porównajmy pozostałe 2 modele.

Modelem z **najwyższą średnią precyzją (*accuracy*)** jest **Random Forest**. **K-NN** jest gorszy jedynie o ok. 1 punkt procentowy. Natomiast plusem **K-NN** jest długość czasu trenowania z predykcją w porównaniu do **Random Forest**. Warto dodać, że czas predykcji **Random Forest** jest zazwyczaj zdecydowanie lepszy (to czas trenowania drzewa wpływa na jego niekorzyść). Oznacza to, że jeśli zdecydujemy się podnieść ilość predykowanych danych ogólny czas może okazać się bez wątpienia lepszy w przypadku **Random Forest**.

Podsumowując różnica w precyzji między **Random Forest**, a **K-NN** nie jest duża, oba modele radzą sobie równie dobrze z rozpoznawaniem liter. Sugerując się czasem stwierdzamy, że **K-NN** jest najlepszy dla tego typu zadań z tą ilością danych.